

C++的STL详细内容（零基础轻松学）

原创

小镇的紫罗兰花园

于 2024-04-17 15:55:18 发布

版权

阅读量718

收藏 15

点赞数 14

文章标签：

c++

开发语言

基本概念

STL（Standard Template Library，标准模版库）是惠普实验室开发的一系列软件的统称。现在主要出现在C++中，但在被引入C++之前该技术就已经存在了很长一段时间。

STL从广义上讲分为三类：

组件	描述
<algorithm>（算法）	容器是用来管理某一类对象的集合。C++ 提供了各种不同类型的容器，比如 deque、list、vector、map 等。
<container>（容器）	算法作用于容器。它们提供了执行各种操作的方式，包括对容器内容执行初始化、排序、搜索和转换等操作。
<iterator>（迭代器）	迭代器用于遍历对象集合的元素。这些集合可能是容器，也可能是容器的子集。

- 容器和算法通过迭代器可以进行无缝衔接。几乎所以代码都采用了模板类和模板函数的方1式，这相比于传统的由函数和类组成的库来说提供了更好的代码重用机会。在C++标准中，STL被组织为下面13个头文件：

<algorithm>、<deque>、<functional>、<iterator>、<vector>、<list>、<map>、<memory>、<numeric>、<queue>、<set>、<stack>、<utility>

1.序列式容器（Sequence containers）

- 每个元素都有固定位置——取决于插入时机和地点，与元素的值无关
- vector、deque、list、stack、queue

2.关联式容器（Associated containers）

- 元素位置取决于特定的排序准则，和插入顺序无关

- set、multiset、map、multimap

数据结构	描述	实现头文件
向量 (vector)	连续存储的元素	<vector>
列表 (list)	由节点组成的双向链表，每个节点包含着一个元素	<list>
双队列 (deque)	连续存储的指向不同元素的指针所组成的数组	<deque>
集合 (set)	由节点组成的红黑树，每个节点都包含着一个元素，节点之间以某种作用于元素对的谓词排列，没有两个不同的元素能够拥有相同的次序	<set>
多重集合 (multiset)	允许存在两个次序相等的元素的集合	<set>
栈 (stack)	后进先出的值的排列	<stack>
队列 (queue)	先进先出的值的排列	<queue>
优先队列 (priority queue)	元素的次序是由作用于所储存的值对上的某种谓词决定的一种队列	<queue>
映射 (map)	又{键，值}对组成的集合，以某种作用于键对上的谓词排列	<map>
多重映射 (multimap)	允许键对有相同的次序映射	<map>

一、迭代器 < iterator^Q >

1、迭代器的基本概念

- **什么是迭代器？**

迭代器是一种检查容器内元素并且遍历容器内元素的**数据类型**

- **迭代器的作用：**

迭代器提供对一个容器中的对象的访问方法，并且定义了容器中对象的范围

- **为什么需要迭代器？**

1. STL提供每种容器的实现原理各不相同，如果没有迭代器，我们需要记住每一种容器中对象的访问方法，很显然这样会变得非常麻烦
2. 每个容器都实现了一个迭代器用于对容器中对象的访问，虽然每个容器中的迭代器的实现方式不一样，但是对于用户来说，操作方式是一致的，也就是说：通过迭代

器统一了对所有容器的访问方式。例如：无论哪个容器访问当前元素的下一个元素，我们都可以通过迭代器自增进行访问

3. 迭代器是为了提高编程效率而开发的

2、迭代器失效

1.插入元素后造成迭代器失效

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      v1.push_back(1);
8      v1.push_back(2);
9      v1.push_back(3);
10     v1.push_back(4);
11
12     vector<int>::iterator it = v1.begin() + 3;
13     cout << *it << endl;
14
15     v1.insert(it, 8);      //会改变容器的位置
16     cout << *it << endl;  //迭代器指向的已不是容器的那个位置，系统会报错
17
18     for (it = v1.begin(); it != v1.end(); it++)
19         cout << *it << " ";
20     cout << endl;
21
22     return 0;
23 }
```

其输出结果为：

```
1  4
2  0
3  1 2 3 8 4
```

其中第二行的结果为0只是因为编译器的结果，因为在给v1容器插入8这个元素的时候，v1容器扩容了，会导致容器的位置发生整体改变，迭代器it所指向的位置就不是原来v1容器的位置了，it变成了一种野指针（使用野指针在VS编译器中会直接报错），也就是**迭代器失效**

insert函数是有返回值的，其返回值是指向插入位置的迭代器

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      v1.push_back(1);
8      v1.push_back(2);
9      v1.push_back(3);
10     v1.push_back(4);
11
12     vector<int>::iterator it = v1.begin() + 3;
13     cout << *it << " ";
14
15     it = v1.insert(it, 8);
16     cout << *it << endl;
17
18     return 0;
19 }
```

此代码输出的结果为：

```
1 | 4 8
```

2.删除元素后造成迭代器失效

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      int a[] = { 1,1,2,3,3,3,3,4,6,6 };
7      vector<int> v1(a, a + 10);
8      vector<int>::iterator it;
9
10     for (it = v1.begin(); it != v1.end(); it++)
11         if (*it == 3)
12             v1.erase(it);
13
14     for (it = v1.begin(); it != v1.end(); it++)
```

```

15         cout << *it << " ";
16     cout << endl;
17
18     return 0;
19 }

```

这段代码输出的结果为：

```

1 | 1 1 2 3 3 4 6 6

```

这时你可能会发现，这个v1容器中的“3”并没有删干净，而是保留了一半，这是为什么呢？

因为容器在删除元素时，会将该元素后面所有的元素整体往前移从而导致一个“3”被“跳过去”了

当然，解决这个问题并不难，只有在迭代器当前的位置不是“3”的时候才使it++，便可实现将“3”去除干净，即：

```

1 //将上述代码的删除“3”操作改为以下这样：
2 for (it = v1.begin(); it != v1.end(); it++)
3     while (*it == 3)
4         v1.erase(it);

```

结果将变为：

```

1 | 1 1 2 4 6 6

```

其实，erase()函数也是有返回值的

```

1 #include<iostream>
2 #include<vector>
3 using namespace std;
4 int main()
5 {
6     int a[] = { 1,1,2,3,3,3,3,4,6,6 };
7     vector<int> v1(a, a + 10);
8     vector<int>::iterator it;
9
10    for (it = v1.begin(); it != v1.end(); )
11        if (*it == 3)
12        {
13            vector<int>::iterator it1;
14            it1 = v1.erase(it);    //此时it这个迭代器会失效

```

```

15         cout << *it1 << " ";
16     }
17     else it++;
18
19     return 0;
20 }

```

其输出的结果为：

```

1 | 3 3 3 4

```

即：返回值是指向所删除位置的迭代器

所以，可以利用这个特性重新使得it指向下一个元素，即删除“3”的完整版代码：

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      int a[] = { 1,1,2,3,3,3,3,4,6,6 };
7      vector<int> v1(a, a + 10);
8      vector<int>::iterator it;
9
10     for (it = v1.begin(); it != v1.end(); )
11         if (*it == 3)
12             it = v1.erase(it);
13         else it++;
14
15     for (it = v1.begin(); it != v1.end(); it++)
16         cout << *it << " ";
17     cout << endl;
18
19     return 0;
20 }

```

代码输出的结果为：

```

1 | 1 1 2 4 6 6

```

3.erase和insert的返回值都是迭代器

erase会返回删除元素的下一个元素的迭代器 当erase（迭代器it）后 it能再使用（被删除了）
因此**在使用erase是要记录返回迭代器的位置** `it = erase（迭代器it）`

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int>v;
7      for (int i = 0; i < 10; i++)
8      {
9          v.push_back(i + 1);
10     }
11     auto it = v.begin();
12     it = v.erase(it);    //如果v.erase(it)后*it报错
13     cout << *it << endl;
14     return 0;
15 }

```

二、容器 <container>

1、vector容器（向量）

1.简介

- vector是将元素置于一个动态数组中加以管理的容器
- vector可以随机存取元素（支持索引值直接存取，用[]操作符或at()的方法）
- vector尾部添加或删除元素非常快速，但是中部或头部插入或删除元素比较费时

2.vector对象的默认构造

vector采用模板类实现，vector对象的默认构造形式

```

1  vector<T> vecT;
2
3  vector<int> vecInt;    //一个存放int类型元素的vector容器
4  vector<float>vecFloat;    //一个存放float类型元素的vector容器
5  vector<string>vecString;    //一个存放string类型元素的vector容器
6
7  class CA{};
8  vector<CA*> vecpCA;    //用于存放类CA的指针的vector容器
9  vector<CA> vecCA;    //用于存放类CA的vector容器
10 //所以此时CA必须提供CA的拷贝构造函数，以保证CA对象之

```

3.vector对象的带参数构造

- `vec(beg, end);` //构造函数将 `[beg, end)` 区间中的元素拷贝给本身。注意该区间是左闭右开区间!
- `vec(n, elem);` //构造函数将 `n` 个 `elem` 拷贝给本身
- `vec(const vector &vec);` //拷贝构造函数

接下来是上述理论的实例:

```

1  | #include<iostream>
2  | #include<vector>
3  | using namespace std;
4  | int main()
5  | {
6  |     int arr[] = { 1,2,3,4,5 };
7  |     vector<int> v1(arr, arr + 5);    //构造形如vector(beg, end);的容器
8  |
9  |     vector<int> v2(3, 10);           //构造形如vector(n, elem);的容器
10 |     for (int i = 0; i < 3; i++)
11 |         cout << v2[i] << " ";
12 |     cout << endl;
13 |
14 |     vector<int> v3(v1);              //构造形如vector(const vector &vec);的容器
15 |     for (int i = 0; i < 5; i++)
16 |         cout << v3[i] << " ";
17 |     return 0;
18 | }
```

输出结果是这样的:

```

1 | 10 10 10
2 | 1 2 3 4 5
```

4.vector的赋值

- `vec.assign(beg, end);` //将 `[beg, end)` 区间中的数据拷贝赋值给本身。注意该区间是左闭右开区间!
- `vec.assign(n, elem);` //将 `n` 个 `elem` 拷贝赋值给本身

注意: 在使用 `assign` 函数时, 会先将容器内的原内容清空, 再赋值。尽管原内容可能比要赋值的内容多

```

1 | #include<iostream>
```

```

2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1(8, 10);
7      for (int i = 0; i < v1.size(); i++)
8          cout << v1[i] << " ";
9      cout << endl;
10
11     int arr[] = { 1,2,3,4,5 };
12     v1.assign(arr, arr + 5);
13     for (int i = 0; i < v1.size(); i++)
14         cout << v1[i] << " ";
15     cout << endl;
16     return 0;
17 }

```

以上代码输出的结果是：

```

1  10 10 10 10 10 10 10 10
2  1 2 3 4 5

```

- vector中的两个迭代器（指针）——begin()和end()

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1, v2, v3, v4;
7      int arr[] = { 1,2,3,4,5 };
8      v1.assign(arr, arr + 5);
9      for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     //用其他容器的迭代器作为参数
14     //begin()指向的是容器中第一个元素
15     //end()指向的是容器中最后一个元素的下一个元素
16     v2.assign(v1.begin(),v1.end());
17     for (int i = 0; i < v2.size(); i++)
18         cout << v2[i] << " ";
19     cout << endl;
20

```

```

21     //如果想实现只将第3个数和第4个数传过去
22     v3.assign(v1.begin() + 2, v1.end() - 1);
23     for (int i = 0; i < v3.size(); i++)
24         cout << v3[i] << " ";
25     cout << endl;
26     return 0;
27 }

```

以上代码输出的结果为：

```

1  1 2 3 4 5
2  1 2 3 4 5
3  3 4

```

- `vec1& operator=(const vector &vec2);` //重载等号操作符
- `vec1.swap(vec2);` //将vec与本身的元素互换

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      int arr[] = { 2,7,8,5,1 };
7      vector<int> v1(arr, arr + 5);
8
9      vector<int> v2, v3;
10     v2 = v1;    //使用重载等号操作符给v2赋值
11     v3.assign(5, 8);
12
13     //分别打印v2, v3容器内的元素
14     cout << "交换之前, v2与v3的值" << endl;
15     for (int i = 0; i < v2.size(); i++)
16         cout << v2[i] << " ";
17     cout << endl;
18     for (int i = 0; i < v3.size(); i++)
19         cout << v3[i] << " ";
20     cout << endl << endl;
21
22     //交换v2与v3的元素
23     v2.swap(v3);    //写成v3.swap(v2);也是一样的
24
25     //再次打印v2, v3容器内的元素
26     cout << "交换之后, v2与v3的值" << endl;
27     for (int i = 0; i < v2.size(); i++)

```

```

28         cout << v2[i] << " ";
29     cout << endl;
30     for (int i = 0; i < v3.size(); i++)
31         cout << v3[i] << " ";
32     cout << endl;
33
34     return 0;
35 }

```

5.vector的大小（长度/个数）

- `vec.size();` //返回vector容器中元素的个数
- `vec.empty();` //判断vector容器是否为空

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      cout << "v1.size:" << v1.size() << endl;
8
9      //判断v1是否为空
10     if (v1.empty())
11         cout << "v1 is empty" << endl;
12     else cout << "v1 is not empty" << endl;
13
14     int arr[] = { 1,2,3,4,5 };
15     v1.assign(arr, arr + 5);
16
17     //给v1赋值之后再次判断v1是否为空
18     cout << "v1.size:" << v1.size() << endl;
19     if (v1.empty())
20         cout << "v1 is not empty" << endl;
21     else cout << "v1 is not empty" << endl;
22
23     return 0;
24 }

```

以上代码的结果是：

```

1  v1.size:0
2  v1 is empty
3  v1.size:5

```

- `vec.resize(num);` //重新指定容器的长度为num。若容器变长，则以默认值重新填充新位置；若容器变短，则末尾超出容器长度的元素被删除。

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      int arr[] = { 1,2,3,4,5 };
8      v1.assign(arr, arr + 5);
9      for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     //将容器的长度增长
14     v1.resize(10);
15     for (int i = 0; i < v1.size(); i++)
16         cout << v1[i] << " ";
17     cout << endl;
18
19     //将容器的长度缩短
20     v1.resize(3);
21     for (int i = 0; i < v1.size(); i++)
22         cout << v1[i] << " ";
23     cout << endl;
24     return 0;
25 }
```

而以上代码运行的结果为：

```

1  1 2 3 4 5
2  1 2 3 4 5 0 0 0 0 0
3  1 2 3
```

也能由此看出在这个编译器（VS）中，`resize()`函数的默认值为0，但更多时候需要将扩展的元素赋为特定的值

- `vec.resize(num, elem);` //重新指定容器的长度为num。若容器变长，则以elem值重新填充新位置；若容器变短，则末尾超出容器长度的元素被删除。

```

1 //只需要将上述resize()函数的num后加上想要的值即可
2 //若将上述代码的增长部分改为以下代码
3 v1.resize(10,100);
4     for (int i = 0; i < v1.size(); i++)
5         cout << v1[i] << " ";
6     cout << endl;

```

则代码的运行结果是这样的：

```

1 1 2 3 4 5
2 1 2 3 4 5 100 100 100 100 100
3 1 2 3

```

6.vector元素的访问方式（数据存取）

vector容器相当于动态数组，长度也是动态的

- vec[idx]; //返回下标(索引)idx所指的数据，越界时，运行直接报错
- vec.at(idx); //返回下标(索引)idx所指的数据，如果idx越界，抛出异常终止的原因（程序仍然会异常终止）

```

1 #include<iostream>
2 #include<vector>
3 using namespace std;
4 int main()
5 {
6     vector<int> v1;
7     int arr[] = { 1,2,3,4,5 };
8     v1.assign(arr, arr + 5);
9     for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     //使用下标访问vector容器中的元素
14     //如果下标越界，可能会导致程序异常终止
15
16     v1[8]=100;        //使用该语句将不知道出错在哪
17
18     v1.at(8) = 100;    //使用该语句将弹出出错的原因
19
20     return 0;
21 }

```

7.在vector末尾的添加移除操作

- `vec.push_back(num);` //在vec的末尾插入一个num元素
- `vec.pop_back();` //删除vec的末尾的一个元素

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      int arr[] = { 1,2,3,4,5 };
8      v1.assign(arr, arr + 5);
9      for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     v1.pop_back();
14     for (int i = 0; i < v1.size(); i++)
15         cout << v1[i] << " ";
16     cout << endl;
17
18     v1.push_back(10);
19     for (int i = 0; i < v1.size(); i++)
20         cout << v1[i] << " ";
21     cout << endl;
22     return 0;
23 }
```

以上代码输出的结果是：

```
1  1 2 3 4 5
2  1 2 3 4
3  1 2 3 4 10
```

8.vector的插入

- `vec.insert(pos, elem);` //在pos位置插入一个elem元素，返回新数据的位置
- `vec.insert(pos, n, elem);` //在pos位置插入n个elem元素，无返回值

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
```

```

4  int main()
5  {
6      vector<int> v1;
7      int arr[] = { 1,2,3,4,5 };
8      v1.assign(arr, arr + 5);
9      for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     //在第3个元素后填入100，之后的元素往后移一个位置
14     v1.insert(v1.begin() + 3, 100);
15     for (int i = 0; i < v1.size(); i++)
16         cout << v1[i] << " ";
17     cout << endl;
18
19     //在第2个元素后填入n个9，之后的元素往后移n个位置
20     v1.insert(v1.begin() + 2, 3, 9);
21     for (int i = 0; i < v1.size(); i++)
22         cout << v1[i] << " ";
23     cout << endl;
24     return 0;
25 }

```

上述代码的结果为：

```

1  1 2 3 4 5
2  1 2 3 100 4 5
3  1 2 9 9 9 3 100 4 5

```

- `vec.insert(pos, beg, end);` //在pos位置插入[beg, end)区间的数据，无返回值

```

1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  int main()
5  {
6      vector<int> v1;
7      int arr[] = { 1,1,1,1,1,1 };
8      v1.assign(arr, arr + 5);
9      for (int i = 0; i < v1.size(); i++)
10         cout << v1[i] << " ";
11     cout << endl;
12
13     int b[] = { 2,3,4,5,6,7 };

```

```

14 //在第2个元素后插入[beg, end)区间，之后的元素往后移n个位置
15 v1.insert(v1.begin() + 2, b + 1, b + 4);
16 for (int i = 0; i < v1.size(); i++)
17     cout << v1[i] << " ";
18 cout << endl;
19 return 0;
20 }

```

上述代码输出的结果为：

```

1 | 1 1 1 1 1
2 | 1 1 3 4 5 1 1 1

```

9.vector容器的iterator（迭代器）类型

- `vector<int>::iterator iter;` //迭代器变量名为iter
- vector容器的迭代器属于“随机访问迭代器”：迭代器一次可以移动多个位置

成员函数	功能
<code>begin()</code>	返回指向容器中第一个元素的正向迭代器；如果是const类型容器，则该函数返回的是常量正向迭代器
<code>end()</code>	返回指向容器最后一个元素的 后一个位置 的正向迭代器；如果是const类型容器，则该函数返回的是常量正向迭代器。此函数通常和 <code>begin()</code> 搭配使用
<code>rbegin()</code>	返回指向最后一个元素的反向迭代器；如果是const类型容器，则该函数返回的是常量反向迭代器
<code>rend()</code>	返回指向容器最后一个元素的 前一个位置 的反向迭代器；如果是const类型容器，则该函数返回的是常量反向迭代器。此函数通常和 <code>rbegin()</code> 搭配使用
<code>cbegin()</code>	和 <code>begin()</code> 功能类似，只不过其返回的迭代器类型为常量正向迭代器，不能用于修改元素
<code>cend()</code>	和 <code>end()</code> 功能类似，只不过其返回的迭代器类型为常量正向迭代器，不能用于修改元素
<code>crbegin()</code>	和 <code>rbegin()</code> 功能类似，只不过其返回的迭代器类型为常量正向迭代器，不能用于修改元素
<code>crend()</code>	和 <code>rend()</code> 功能类似，只不过其返回的迭代器类型为常量正向迭代器，不能用于修改元素

```

1 | #include<iostream>
2 | #include<vector>
3 | using namespace std;

```

```

4  int main()
5  {
6      vector<int> v1;
7      int iarray[] = { 100,1,20,30,40 };
8
9      v1.assign(iarray, iarray + 5);
10
11     //构造一个迭代器对象it
12     vector<int>::iterator it;
13     // 让迭代器it指向v1容器的第一个元素
14     it = v1.begin();
15
16     //输出此时的it指向的元素
17     cout << *it << endl;
18
19     //通过循环使用迭代器遍历v1容器中的所有元素
20     for (it = v1.begin(); it != v1.end(); it++)
21         cout << *it << " ";
22     cout << endl;
23
24     it = v1.begin();
25     it = it + 2;
26     cout << *it;
27
28     return 0;
29 }

```

此时的输出结果为：

```

1 | 100
2 | 100 1 20 30 40
3 | 20

```

2、deque容器（双端队列）

1.deque容器简介

- deque是“double-ended queue”的缩写，和vector一样都是STL的容器
- deque是双端数组而vector是单端的
- deque在接口上和vector非常相似，在许多操作的地方可以直接替换
- deque可以随机存取元素（支持索引值直接存取，用[]操作符或at()方法）
- deque头部和尾部添加或移除元素都非常快速。但是在中部安插元素或移除元素比较费

时

- 需要使用头文件#include<deque>

2.deque容器的操作

deque与vector在操作上几乎一样，deque多两个函数

```
1 | #include<deque>
2 | deque.push_front(elem);      //在容器头部插入一个数据
3 | deque.pop_front();           //删除容器的第一个数据
```

以下是在头部插入数据的实例：

```
1 | #include<iostream>
2 | #include<deque>
3 | using namespace std;
4 | int main()
5 | {
6 |     deque<int> d1 = { 10,20,30,40 };
7 |
8 |     //在头部插入一个元素
9 |     d.push_front(100);
10 |    deque<int>::iterator it;
11 |    for (it = d1.begin(); it != d1.end(); it++)
12 |        cout << *it << " ";
13 |    cout << endl;
14 |    return 0;
15 | }
```

以上代码输出的结果为：

```
1 | 100 10 20 30 40
```

3、stack容器（栈）

1.stack对象的默认构造

- stack采用模板类实现，stack对象的默认构造形式为：**stack<T> s;**
- stack<int> s; //一个存放int的stack容器
- stack<float> s; //一个存放float的stack容器

- `stack<string> s;` //一个存放string的stack容器

2.stack常用的成员函数

- `stack.push(elem)` //在栈顶增加元素（入栈）
- `stack.pop()` //移除栈顶元素（出栈）
- `stack.empty()` //堆栈为空则返回真
- `stack.size()` //返回栈中元素数目
- `stack.top()` //返回栈顶元素

```
1  #include<iostream>
2  #include<stack>
3  using namespace std;
4  int main()
5  {
6      //以默认构造方式构造一个stack容器
7      stack<int> s;
8
9      //入栈
10     for(int i=1;i<8;i+=2)
11         s.push(i);
12
13     //stack容器没有迭代器，因为栈是不允许遍历的
14     cout<<s.top()<<" "; //top函数返回栈顶元素，不会删除栈顶元素
15     s.pop();           //出栈，删除栈顶元素
16     cout<<s.top();
17
18     return 0;
19 }
```

以上代码输出的结果为：

```
1 | 7 5
```

栈空之后就不能继续移除栈顶元素了，我们通常会选择通过判断栈是否为空来作为是否进行的条件，当然，在栈空之后再使用`pop()`函数是很危险的

```
1  #include<iostream>
2  #include<stack>
3  using namespace std;
4  int main()
```

```

5  {
6      stack<int> s;
7      for (int i = 1; i < 10; i += 2)
8          s.push(i);
9
10     //可以通过while来显示并清除栈内所有元素
11     //此时，由于栈是“先入后出”的模式，所以会以与入栈时相反的顺序出栈
12     while(!s.empty())    //或者写成s.empty()==false的形式
13     {
14         cout<<s.top()<<" ";
15         s.pop();
16     }
17     return 0;
18 }

```

以上代码输出的结果为：

```

1 | 9 7 5 3 1

```

3.stack对象的拷贝构造与赋值

- stack(const stack &stk); //拷贝构造函数
- stack& operator = (const stack &stk); //重载等号操作符

```

1  #include<iostream>
2  #include<stack>
3  using namespace std;
4  int main()
5  {
6      stack<int> stk1;
7      for (int i = 1; i < 10; i += 2)
8          stk1.push(i);
9      stack<int> stk2(stk1);    //将stk1拷贝到skt2中
10     while (!stk2.empty())
11     {
12         cout << stk2.top() << " ";
13         stk2.pop();
14     }
15     return 0;
16 }

```

以上代码输出的结果为：

```
1 | 9 7 5 3 1
```

很显然，是和stk1的结果是一样的。但对于“=”，在定义时使用和在语句中使用的意义是不一样的

```
1 | //接上一条代码
2 | stack<int> stk3 = stk; //调用拷贝构造函数，与上述stk2是等价的
3 | stk3 = stk1; //此时调用了“=”的重载功能
```

4.stack容器的大小（栈内元素的个数）

```
1 | #include<iostream>
2 | #include<stack>
3 | using namespace std;
4 | int main()
5 | {
6 |     stack<int> stk1;
7 |     for (int i = 1; i < 10; i += 2)
8 |         stk1.push(i);
9 |     stack<int> stk2(stk1); //将stk1拷贝到stk2中
10 |    while (!stk2.empty())
11 |    {
12 |        cout << stk2.top() << " ";
13 |        stk2.pop();
14 |    }
15 |    cout << endl;
16 |    cout << "stk1 size:" << stk1.size() << endl;
17 |    return 0;
18 | }
```

上述代码输出的结果为：

```
1 | 9 7 5 3 1
2 | stk1 size:5
```

stack容器并没有提供resize()方法

4、queue容器（队列）

- queue是队列容器，是一种“先进先出”的容器
- 需要使用头文件#include<queue>

1.queue对象的默认构造

- queue采用模板类，queue对象的默认构造形式：queue<T> q;

```
1 | #include<queue>
2 | #include<cstring>
3 | queue<int> queInt;      //一个存放int的queue容器
4 | queue<float> queFloat;  //一个存放float的queue容器
5 | queue<string> queString; //一个存放string的queue容器
```

2.queue容器的常用成员函数

- queue.push(elem) //在队列尾部增加元素（入队）
- queue.pop() //删除队首元素（出队）
- queue.empty() //队列为空则返回真
- queue.size() //返回队列中元素数目
- queue.front() //返回队首元素（可以作为左值被修改）
- queue.back() //返回队尾元素

```
1 | #include<iostream>
2 | #include<queue>
3 | using namespace std;
4 | int main()
5 | {
6 |     //默认构造方式构造一个队列容器
7 |     queue<int> q1;
8 |
9 |     //向队列中添加元素
10 |    for (int i = 1; i < 10; i += 2)
11 |        q1.push(i);
12 |
13 |    //queue容器也没有提供迭代器，因为不能够遍历
14 |    while(!q1.empty())
15 |    {
16 |        cout << q1.front() << endl;
17 |        q1.pop();    //删除第一个元素（队首元素）
18 |    }
19 |    return 0;
20 | }
```

3.queue容器对象的拷贝构造与赋值

- `queue(const queue &que);` //拷贝构造函数
- `queue& operator = (const queue &que);` //重载等号操作符

基本上与stack容器相同，不再赘述了

5、list容器（双向链表）

慎用STL的list，空间复杂度和时间复杂度都容易超出限制

1.基本概念

- list是一个双向链表容器，可高效地进行插入和删除元素
- list不可以随机存取元素，所以不支持`at(pos)`函数与`[]`操作符
 - `it++` //迭代器自增、自减是正确的
 - `it+5` //迭代器一次移动多个是错误的
- 需要使用`#include<list>`头文件

2.list容器的头部和尾部操作

- list采用模板类实现，对象的默认构造形式：`list<T> lst`，例如：

```
1 | list<int> lstInt;    //定义一个存放int的list容器
2 | list<float> lstFloat; //定义一个存放float的list容器
3 | list<string> lstString; //定义一个存放string的list容器
```

list头尾添加移除操作

- `list.push_back(elem);` //在容器尾部加入一个元素
- `list.pop_back( );` //删除容器中最后一个元素
- `list.push_front(elem);` //在容器头部加入一个元素
- `list.pop_front( );` //删除容器中第一个元素

```
1 | #include<iostream>
2 | #include<list>
3 | using namespace std;
4 | int main()
5 | {
```

```

6     list<int> lst;
7
8     lst.push_back(10); //在尾部插入一个元素（类似尾插法）
9     lst.push_front(20); //在头部插入一个元素（类似头插法）
10
11    //构造一个迭代器
12    list<int>::iterator it;
13    for (it = lst.begin(); it != lst.end(); it++)
14        cout << *it << " ";
15    cout << endl;
16
17    //删除最后一个元素
18    lst.pop_back();
19    for (it = lst.begin(); it != lst.end(); it++)
20        cout << *it << " ";
21    cout << endl;
22    lst.push_back(10); //把刚才删除的元素加回来
23
24    //删除第一个元素
25    lst.pop_front();
26    for (it = lst.begin(); it != lst.end(); it++)
27        cout << *it << " ";
28    cout << endl;
29    lst.push_front(20); //把刚才删除的元素加回来
30
31    //返回第一个节点
32    int x = lst.front();
33    cout << "front:" << x << endl;
34
35    //返回最后一个节点
36    x = lst.back();
37    cout << "back:" << x << endl;
38
39    //list容器是可遍历，内部数据同样是可改变的
40    lst.front() = 100; //将第一个节点赋值为100
41    lst.back() = 200; //将最后一个节点赋值为200
42    for (it = lst.begin(); it != lst.end(); it++)
43        cout << *it << " ";
44    cout << endl;
45    return 0;
46 }

```

以上代码输出的结果为：

```

1 | 20 10
2 | 20

```

```
3 | 10
4 | front:20
5 | back:10
6 | 100 200
```

3.list的反向迭代器

- list容器的迭代器是“双向迭代器”：双向迭代器从两个方向读写容器。除了提供前向迭代器的全部操作之外，双向迭代器还提供前置和后置的自增、自减运算

```
1 | list.begin();    //返回容器中第一个元素的迭代器
2 | list.end();      //返回容器中最后一个元素之后的迭代器
3 | list.rbegin();   //返回容器中倒数第一个元素的迭代器
4 | list.rend();     //返回容器中倒数最后一个元素的后面的迭代器
```

注意，不论正向迭代器还是反向迭代器，it++的方向都是从begin()到end()

```
1 | #include<iostream>
2 | #include<list>
3 | using namespace std;
4 | int main()
5 | {
6 |     list<int> lst;
7 |     lst.push_back(10);
8 |     lst.push_front(20);
9 |     lst.push_back(30);
10 |    lst.push_back(40);
11 |    //20 10 30 40
12 |
13 |    //list容器的反向迭代器
14 |    list<int>::reverse_iterator it1;
15 |    for (it1 = lst.rbegin(); it1 != lst.rend(); it1++)
16 |        cout << *it1 << " ";
17 |    cout << endl;
18 |    return 0;
19 | }
```

以上代码输出的结果为：

```
1 | 40 30 10 20
```

同时证明了list是一个双向链表

4. list对象的带参数构造

- `list(n,elem);` //构造函数将n个elem拷贝给本身
- `list(beg,end);` //构造函数将[beg,end)区间中的元素拷贝给本身
- `list(const list &lst);` //拷贝构造函数

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4  int main()
5  {
6      list<int> lst1(3, 5);    //将
7      list<int>::iterator it;
8      for (it = lst1.begin(); it != lst1.end(); it++)
9          cout << *it << " ";
10     cout << endl;
11
12     list<int> lst;
13     lst.push_back(10);
14     lst.push_front(20);
15     lst.push_back(30);
16     lst.push_back(40);
17     //20 10 30 40
18
19     //list不支持+n的情况存在
20     //list<int> lst2(lst.begin(),lst.begin()+2);是错误的
21     list<int>::iterator beg = lst.begin();
22     list<int>::iterator end = lst.begin();
23     end++; end++;
24     for (it = beg; it != end; it++)
25         cout << *it << " ";
26     cout << endl;
27
28     list<int> lst3(lst);      //拷贝构造函数
29     for (it = lst3.begin(); it != lst3.end(); it++)
30         cout << *it << " ";
31     cout << endl;
32     return 0;
33 }
```

以上输出的结果为:

```
1 | 5 5 5
2 | 20 10
```

5.list容器的赋值

- `list.assign(beg,end);` //将[beg,end)区间中的数据拷贝赋值给本身
- `list.assign(n,elem);` //将n个elem拷贝赋值给本身
- `list& operator = (const list &lst);` //重载等号操作符
- `list.swap(lst);` //交换两个头指针的位置，类似于交换元素

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4  int main()
5  {
6      list<int> lst;
7      lst.push_back(10);
8      lst.push_front(20);
9      lst.push_back(30);
10     lst.push_back(40);
11     //20 10 30 40
12
13     list<int> lst1;
14     lst1.assign(lst.begin(), lst.end());
15     list<int>::iterator it;
16     for (it = lst1.begin(); it != lst1.end(); it++)
17         cout << *it << " ";
18     cout<<endl;
19
20     lst1.assign(3,6);    //覆盖式赋值（会清空赋值之前的元素），而不是追加
21     for (it = lst1.begin(); it != lst1.end(); it++)
22         cout << *it << " ";
23
24     list<int> lst2(4,8);
25     lst2.swap(lst1);    //交换两个头指针的位置
26     for (it = lst1.begin(); it != lst1.end(); it++)
27         cout << *it << " ";
28     cout<<endl;
29     for (it = lst2.begin(); it != lst2.end(); it++)
30         cout << *it << " ";
31     cout<<endl;
32     return 0;
33 }
```

以上代码输出的结果为：

```
1 | 20 10 30 40
2 | 6 6 6
3 | 8 8 8 8
4 | 6 6 6
```

6.list的成员函数

(1)专用成员函数

- merge(b)：将链表b与调用链表合并，在合并之前，两个链表必须已经排序，合并后经过排序的链表被保存在调用链表中，b为空。
- remove(val)：从链表中删除val的所有节点。
- splice(pos,b)：将链表b的内容插入pos的前面，b为空。
- reverse()：将链表翻转。
- sort()：将链表排序。
- unique()：将连续的相同元素压缩为单个元素。不连续的相同元素无法压缩，因此一般先排序后去重。

(2)其他成员函数

- push_front(x)/push_back(x)：x从链表头或尾入。
- pop_front()/pop_back()：从链表头或尾出。
- front()/back()：返回链表头或尾元素。
- insert(p, t)：在p之前插入t。
- erase§：删除p。
- clear()：清空链表。

6、set/multiset容器（集合）

1.set/multiset容器的基本概念

- set是一个集合容器，其中所包含的元素是唯一的，集合中的元素按一定的顺序排列。元素插入过程是按排序规则插入，所以不能指定插入位置
- set采用**红黑树**变体的数据结构实现，红黑树属于平衡二叉树。在插入操作和删除操作上比vector快

- set不可以直接存取元素（不可以使用at.(pos)与[]操作符）
- multiset与set的区别：set支持唯一键值，每个元素值只能出现一次；而multiset中同一值可以出现多次
- 不可以直接修改set或multiset容器中的元素值，因为该类容器是自动排序的。如果希望修改一个元素值，必须先删除原有的元素，再插入新的元素
- set或multiset的迭代器为双向访问，不支持随机访问。执行一次“++”和“- -”操作的时间复杂度均为 $O(\log n)$ 。默认的元素顺序为升序，也可以通过第2个模版的参数设置为降序
- 头文件为#include<set>

2.set容器的插入和迭代器

- set.insert(elem); //在容器中插入元素
- set.begin(); //返回容器中第一个数据的迭代器
- set.end(); //返回容器中最后一个数据之后的迭代器
- set.rbegin(); //返回容器中倒数第一个元素的迭代器
- set.rend(); //返回容器中倒数最后一个元素的后面的迭代器

```

1  #include<iostream>
2  #include<set>
3  using namespace std;
4  int main()
5  {
6      set<int> s1;
7      s1.insert(3);
8      s1.insert(1);
9      s1.insert(5);
10     s1.insert(2);
11     set<int>::iterator it;
12     for (it = s1.begin(); it != s1.end(); it++)
13         cout << *it << ' ';
14     cout << endl;
15     return 0;
16 }
```

上述代码输出的结果为：

```
1 | 1 2 3 5
```

由此可知，set容器的默认排序是**升序**排列的

如果想让set按照降序的方式进行排序，则可以按照以下方式达成：

```
1 //将上述的 set<int> s1; 改为 set<int,greater<int> > s1;
2 //注意greater<int>后面有空格，避免两个符号一起>>，有右移歧义
3 #include<iostream>
4 #include<set>
5 using namespace std;
6 int main()
7 {
8     set<int,greater<int> > s1;
9     s1.insert(3);
10    s1.insert(1);
11    s1.insert(5);
12    s1.insert(2);
13    set<int>::iterator it;
14    for (it = s1.begin(); it != s1.end(); it++)
15        cout << *it << ' ';
16    cout << endl;
17    return 0;
18 }
```

上述代码输出的结果则为：

```
1 | 5 3 2 1
```

3.set的成员函数

- size/empty/clear：元素个数、判空、清空
- begin/end：开始位置和结束位置
- insert(x)：将元素x插入集合
- erase(x)：删除所有等于x的元素
- erase(it)：删除it迭代器指向的元素
- find(x)：查找元素x在集合中的位置，若不存在，则返回end
- count(x)：统计等于x的元素个数
- lower_bound(x)/upper_bound(x)：返回大于或等于x的最小元素位置、大于x的最小元素位置

7、map/multiset容器（映射）

1.map/multiset的基本概念

map的键和值可以是不同的类型，键是唯一的，每个键都对应一个值。multimap与map类似，只是允许一个键对应多个值。map可被当作哈希表使用，它建立了从键（关键字）到值的映射。map是键和值的一一映射，multimap是一对多映射。使用map或multimap时需要引入头文件#include<map>

map的迭代器和set类似，支持双向访问，不支持随机访问，执行一次“++”和“- -”操作的时间复杂度 $O(\log n)$ 。默认的元素顺序为升序，也可以通过第3个模板参数设置为降序。

```
1 | map<string, int> a;      //升序
2 | map<string, int, greater<string> >a;    //降序
```

上述map模板的第1个参数为键的类型，第2个参数为值的类型，第3个参数可选，用于对键进行排序的比较函数或对象。

在map中，键和值是一对数，可以使用make_pair生成一对数（键，值）进行插入。

```
1 | a.insert(make_pair(s,i));
```

输出时，可以分别输出第1个元素（键）和第2个元素（值）。

```
1 | map<string, int>::iterator it;
2 | for (it = a.begin(); it != a.end(); it++)
3 |     cout << it->first << "\t" << it->second << endl;
```

2.map的成员函数

- size/empty/clear：元素个数、判空、清空
- begin/end：开始位置和结束位置
- insert(x)：将元素x插入集合（x为二元组）
- erase(x)：删除所有等于x的元素（x为二元组）
- erase(it)：删除it指向的元素（it为指向二元组的迭代器）
- find(k)：查找键位k的二元组的位置，若不存在，则返回尾指针

可以通过“[]”操作符直接得到键映射的值，也可以通过赋值操作改变键映射的值，例如h[key]=val。是不是特别像哈希表？

例如，可以用map统计字符串出现的次数。

```
1 map<string, int> mp;
2 string word;
3 for (int i = 0; i < n; i++)
4 {
5     cin >> s;
6     mp[s]++;
7 }
8 cout << "输入字符串s，查询该字符串出现的次数：" << endl;
9 cin >> s;
10 cout << mp[s] << endl;
```

需要特别注意的是，如果查找的key不存在，则执行h[key]之后会自动新建一个二元组(key, 0)并返回0，进行多次查找之后有可能包含很多无用的二元组。因此使用查找时最好先查询key是否存在。

```
1 if (mp.find(s) != mp.end())
2     cout << mp[s] << endl;
3 else
4     cout << "没找到!" << endl;
```

3.multimap的基本阐述

multimap和map类似，不同的是一个键可以对应多个值。由于是一对多的映射关系，multimap不能使用“[]”操作符。

例如，可以添加多个关于X国的数据：

```
1 multimap<string, int> mp;
2 string s1("X"), s2("Y");
3 mpinsert(make_pair(s1, 50));
4 mpinsert(make_pair(s1, 55));
5 mpinsert(make_pair(s1, 60));
6 mpinsert(make_pair(s2, 30));
7 mpinsert(make_pair(s2, 20));
8 mpinsert(make_pair(s1, 10));
```

输出所有关于X国的数据：

```
1 multimap<string, int>::iterator it;
2 it = mp.find(s1);
3 for (int k = 0; k < mp.count(s1); k++, it++)
4     cout << it->first << "--" << it->second << endl;
```

